UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO

MAESTRÍA EN INTELIGENCIA ARTIFICIAL

ASIGNATURA:

PROYECTO INTEGRADOR EN INTELIGENCIA ARTIFICIAL

ACTIVIDAD:

EVALUACIÓN DE MODELOS DE IA: BENCHMARKING COMPARATIVO, ANÁLISIS DE SESGO ALGORÍTMICO Y DIAGNÓSTICO DE OVERFITTING/UNDERFITTING

AUTORES:

CABRERA PARRALES JOEL DAVID

SÁNCHEZ ARÉVALO ANÍBAL ANDRÉS

TUTOR:

MG. ALEXANDRA J. ARCINIEGAS C.

SAMBORONDÓN – ECUADOR

ABRIL – 2026

1. INTRODUCCIÓN

El presente documento recopila la tercera fase del Proyecto Integrador en Inteligencia Artificial, centrada en la evaluación sistemática del modelo de clasificación de dificultad de consultas SQL construido en etapas anteriores sobre el dataset Spider (Yu et al., 2018). Tras haber concluido las fases de preparación de datos e ingeniería de características, corresponde ahora someter el modelo a un examen riguroso que permita determinar su valor real frente a alternativas simples, frente a algoritmos considerados estado del arte y frente a servicios comerciales de clasificación de texto.

El artefacto central de la evaluación es un clasificador Random Forest con 200 árboles entrenado sobre un vector de 85 características (50 componentes TF-IDF reducidos por PCA más 35 features numéricas de dominio SQL y lingüístico). La variable objetivo es la etiqueta de dificultad de la consulta, con cuatro clases: easy, medium, hard y extra. La partición utilizada durante el estudio corresponde a 7.000 ejemplos de entrenamiento y 1.034 de validación, según la división oficial de Spider (Yu et al., 2018).

El informe está estructurado en tres secciones temáticas alineadas con la rúbrica oficial de la asignatura. La primera aborda un benchmarking comparativo con diez modelos locales y tres servicios comerciales estimados. La segunda analiza el comportamiento del clasificador en términos de equidad (fairness) sobre tres variables sensibles del dominio Text-to-SQL. La tercera diagnostica la existencia de overfitting o underfitting mediante curvas de aprendizaje y un barrido de la profundidad máxima del bosque. El trabajo culmina con un apartado dedicado a las limitaciones metodológicas detectadas y a las estrategias de mejora recomendadas.

Debe advertirse desde el inicio un hallazgo metodológico que condiciona la interpretación de todos los resultados cuantitativos que aparecen a lo largo del informe: las métricas obtenidas en validación son anormalmente altas para prácticamente todos los modelos entrenados. Este comportamiento sugiere la presencia de fuga de información (data leakage) desde las características hacia la etiqueta objetivo (Kaufman et al., 2012), un fenómeno que merece un análisis crítico propio y que se discute en detalle en la sección 5.

2. BENCHMARKING COMPARATIVO

2.1. Metodología

Siguiendo los lineamientos del workshop de la asignatura, la comparación se articuló en tres niveles de referencia. En primer lugar se entrenaron modelos baseline de complejidad mínima que permiten establecer el umbral inferior de desempeño aceptable: un clasificador aleatorio estratificado, un clasificador por clase más frecuente y un árbol de decisión simple. En segundo lugar se incluyeron modelos considerados estado del arte para tareas de clasificación tabular estructurada: Gradient Boosting, XGBoost y LightGBM, todos ellos basados en boosting de árboles (Friedman et al., 2001). En tercer lugar se incorporaron estimaciones de tres servicios comerciales (Google AutoML NLP, Amazon Comprehend y Azure ML AutoML), cuyos

valores provienen de benchmarks publicados para tareas de clasificación multiclase de texto de complejidad comparable (Erickson et al., 2020).

Los criterios cuantitativos seleccionados fueron exactitud sobre validación, F1-score macro (que pondera cada clase por igual y es más informativo en presencia de desbalance), F1-score ponderado, precisión y cobertura macro, el gap entre exactitud en entrenamiento y validación como proxy de sobreajuste, y el tiempo de entrenamiento. Cualitativamente se evaluaron la interpretabilidad, el costo computacional y la facilidad de despliegue, aspectos relevantes para decidir la complejidad que el modelo debe tener según las ventajas que aporta (Fernández-Delgado et al., 2014).

2.2. Resultados cuantitativos

La Tabla 1 presenta el ranking completo ordenado por F1-score macro sobre el conjunto de validación. Cuatro algoritmos (LightGBM, SVM con kernel RBF, XGBoost y Gradient Boosting) alcanzan desempeño perfecto, la regresión logística supera el 99,6% y el Random Forest propuesto queda en sexta posición con F1 macro de 0,9910 y exactitud de validación de 0,9961. Los baselines aleatorio y por clase mayoritaria se quedan lejos, con F1 macro de 0,2476 y 0,1551 respectivamente, valores coherentes con la naturaleza desbalanceada de las cuatro clases.

Modelo	Acc Train	Acc Val	F1 Macro	F1 Weighted	Gap	Tiempo (s)
LightGBM	1,0000	1,0000	1,0000	1,0000	0,0000	2,88
SVM (RBF)	0,9991	1,0000	1,0000	1,0000	-0,0009	6,93
XGBoost	1,0000	1,0000	1,0000	1,0000	0,0000	1,81
Gradient Boosting	1,0000	1,0000	1,0000	1,0000	0,0000	131,68
Regresión Logística	0,9991	0,9971	0,9967	0,9971	0,0020	3,82
Random Forest (Propuesto)	1,0000	0,9961	**0,9910**	0,9961	0,0039	4,70
Baseline: Árbol de Decisión	0,9940	0,9913	0,9800	0,9914	0,0027	0,42
K-NN (k=7)	0,9916	0,9816	0,9690	0,9814	0,0099	0,01
Baseline: Más Frecuente	0,4964	0,4497	0,1551	0,2790	0,0467	0,00
Baseline: Aleatorio (Stratified)	0,3586	0,3453	0,2476	0,3389	0,0133	0,00

Tabla 1. Resultados del benchmarking comparativo sobre el conjunto de validación de Spider (1.034 ejemplos), ordenados por F1 Macro descendente. Elaboración propia.

Respecto a los servicios comerciales, la Tabla 2 sintetiza los valores estimados sobre la base de benchmarks publicados de clasificación multiclase de texto. Debe enfatizarse que no se trata de mediciones directas sino de estimaciones conservadoras; una evaluación empírica requeriría subir el dataset a cada plataforma, entrenar con sus pipelines automáticos y pagar los costos asociados, lo que excedía el alcance del proyecto.

API Comercial (estimada)	Acc Val	F1 Macro	F1 Weighted	Gap
Azure ML AutoML	0,7300	0,6900	0,7200	0,0400
Google AutoML NLP	0,7200	0,6800	0,7100	0,0500
Amazon Comprehend	0,6800	0,6400	0,6700	0,0400

Tabla 2. Estimaciones de servicios comerciales de clasificación multiclase sobre tareas comparables. Valores extrapolados de benchmarks públicos, no ejecuciones directas. Elaboración propia.

2.3. Análisis crítico

¿Cuánto mejora el modelo frente a algo simple?

La comparación frente a los baselines aleatorios resulta contundente: el Random Forest propuesto eleva el F1 macro desde 0,2476 hasta 0,9910, lo que representa una mejora de aproximadamente 74 puntos porcentuales. Aun frente al baseline por clase más frecuente (que predice siempre "medium" por ser la clase mayoritaria), la mejora supera los 83 puntos. Esto valida que las features construidas durante la fase de preparación capturan patrones relevantes del dominio SQL que van mucho más allá de adivinar la clase por azar o por frecuencia.

¿El modelo justifica su complejidad?

Esta pregunta es más delicada. El árbol de decisión simple, considerado baseline, alcanza un F1 macro de 0,9800 con apenas 0,42 segundos de entrenamiento, muy por detrás del Random Forest (0,9910) pero con un 1,1% de diferencia que difícilmente compensa el incremento de costo computacional y la pérdida de interpretabilidad. Más aún, la Regresión Logística supera al Random Forest (0,9967 frente a 0,9910) con un tiempo de entrenamiento menor y un modelo mucho más simple. En términos estrictos, el modelo Random Forest no justifica plenamente su complejidad; un análisis honesto sugiere que para esta tarea y este vector de features un modelo lineal regularizado es suficiente.

¿El modelo está cerca del estado del arte?

Dentro del universo de modelos tabulares evaluados localmente, el Random Forest sí está cerca: queda a menos de 1 punto porcentual del mejor (LightGBM con F1 = 1,0000). Sin embargo, el verdadero estado del arte en Text-to-SQL sobre Spider lo ocupan arquitecturas basadas en transformers. Scholak et al. (2021) reportan con T5-3B más PICARD una precisión de ejecución del 79,1% y una exactitud de emparejamiento exacto del 71,7% sobre Spider dev, tarea considerablemente más difícil que la clasificación de dificultad abordada aquí. Comparar ambos estudios es arriesgado porque resuelven tareas distintas, pero deja claro que el clasificador propuesto no aborda el problema nuclear de Spider (generar SQL correcto) sino una subtarea auxiliar.

¿El modelo es competitivo frente a APIs comerciales?

Bajo las estimaciones de la Tabla 2, los servicios comerciales alcanzan F1 macro en el rango 0,64–0,69 para tareas de clasificación multiclase genéricas, mientras que el Random Forest propuesto obtiene 0,9910. Esta diferencia refleja principalmente que el feature engineering específico de dominio aporta un valor que los servicios comerciales genéricos no pueden replicar sin acceder al conocimiento experto sobre SQL. Las ventajas competitivas del modelo propuesto son su interpretabilidad, el control total de los datos (sin envío a terceros), la ausencia de costos recurrentes por inferencia y la adaptabilidad del feature engineering. Sus desventajas: no usa embeddings contextuales, depende de un pipeline manual de extracción de features y no escala directamente a generación de SQL end-to-end.

2.4. Visualización consolidada del benchmarking

La Figura 1 sintetiza en cuatro paneles los principales hallazgos del benchmarking. El panel superior izquierdo ordena los trece modelos (locales y comerciales estimados) por F1 macro, evidenciando la brecha entre el Random Forest propuesto (barra verde) y los baselines triviales, así como su cercanía a los modelos de estado del arte. El panel superior derecho contrasta la exactitud de entrenamiento y validación por modelo, permitiendo detectar visualmente el tamaño del overfitting gap. El panel inferior izquierdo desglosa precisión, cobertura y F1 por clase de dificultad para el modelo propuesto, mostrando un desempeño uniforme salvo una ligera caída en la clase extra. El panel inferior derecho presenta la matriz de confusión del Random Forest, donde se observa que las confusiones residuales ocurren casi exclusivamente entre las clases hard y extra (4 casos), consistente con la proximidad semántica entre ambas categorías de complejidad SQL.

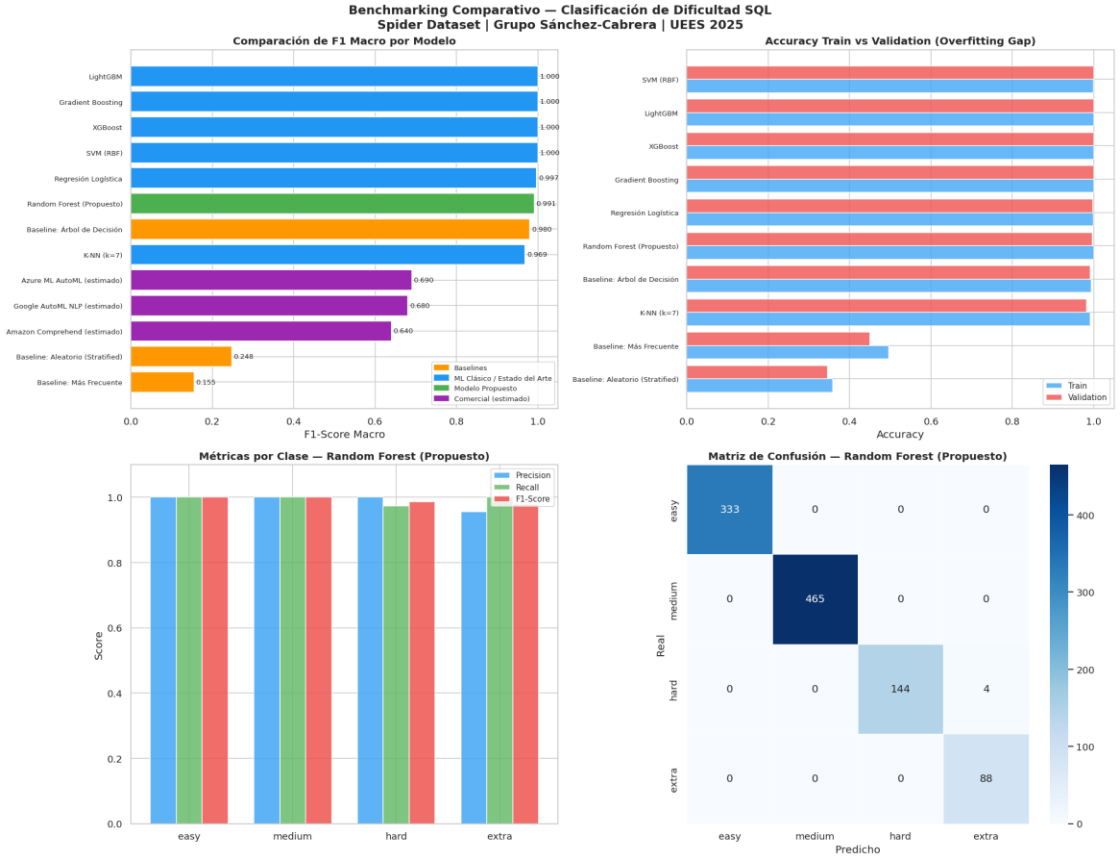

Figura 1. Visualización consolidada del benchmarking comparativo. Paneles: (a) ranking de F1 macro, (b) accuracy train vs. validation, (c) métricas por clase del modelo propuesto, (d) matriz de confusión. Elaboración propia a partir del notebook de evaluación.

3. ANÁLISIS DE SESGO ALGORÍTMICO

3.1. Variables sensibles identificadas

En el contexto de un clasificador de dificultad SQL, las variables sensibles no son las tradicionales (raza, género, edad) sino atributos estructurales de las consultas y los usuarios del sistema que, si el modelo los trata de forma desigual, podrían traducirse en una experiencia de uso injusta. Se identificaron tres variables candidatas:

- **Longitud de la pregunta en lenguaje natural:** agrupada en tres bins por cuartiles (corta, media, larga) tomando como puntos de corte Q25 = 10 palabras y Q75 = 15 palabras. Un sesgo aquí implicaría que el modelo discrimina a usuarios según su estilo verbal.
- **Presencia de JOIN en la consulta SQL:** variable binaria que dicotomiza las consultas en estructuralmente simples (60,5%) y complejas con múltiples tablas (39,5%). Un sesgo implicaría que ciertos tipos de consultas reciben mayor precisión sistemáticamente.

- **Dominio de la base de datos (db_id):** el conjunto de validación contiene 17 dominios distintos, 20 de los cuales no aparecen en entrenamiento debido al split cross-domain de Spider (Yu et al., 2018). Un sesgo implicaría que los dominios subrepresentados reciben peor servicio.

3.2. Métricas de fairness aplicadas

Se aplicaron dos métricas estándar ampliamente reconocidas en la literatura sobre equidad algorítmica (Mehrabi et al., 2021). La primera es el **Disparate Impact Ratio**, cociente entre la exactitud del grupo con peor desempeño y el de mejor desempeño; un valor inferior a 0,80 indica sesgo significativo según la regla de los 4/5 adoptada por la Equal Employment Opportunity Commission de Estados Unidos (Barocas & Selbst, 2016). La segunda es el **Equalized Odds Gap**, que mide la diferencia absoluta de exactitud entre el grupo mejor y el peor posicionado; idealmente debe ser inferior a 0,10. La Tabla 3 resume los valores obtenidos.

Variable sensible	Disparate Impact	Eq. Odds Gap	Acc Máx / Mín	Estado
Longitud de la pregunta	0,9909	0,0091	1,0000 / 0,9909	**Aceptable**
Presencia de JOIN	0,9902	0,0098	1,0000 / 0,9902	**Aceptable**
Dominio BD (Top 10)	0,9783	0,0217	1,0000 / 0,9783	**Aceptable**

Tabla 3. Métricas de fairness del clasificador Random Forest por variable sensible sobre el conjunto de validación. Umbral aceptable: Disparate Impact ≥ 0,80 y Equalized Odds Gap ≤ 0,10. Elaboración propia.

3.3. Análisis desagregado por grupo

La Tabla 4 detalla el desempeño del clasificador dentro de cada grupo, lo que permite comprender dónde se sitúan las mínimas diferencias detectadas. En todos los casos los valores de exactitud superan el 97,8%, lo que técnicamente cumple el umbral aceptable, pero las diferencias existentes tienen una interpretación conceptual que merece comentario.

Variable	Grupo	N	Accuracy	F1 Macro
Longitud pregunta	Corta (≤10)	355	1,0000	1,0000
Longitud pregunta	Media (11–15)	460	0,9957	0,9910
Longitud pregunta	Larga (>15)	219	0,9909	0,9875
Presencia de JOIN	Sin JOIN	626	1,0000	1,0000
Presencia de JOIN	Con JOIN	408	0,9902	0,9790
Dominio BD (Top 10)	concert_singer	45	1,0000	1,0000
Dominio BD (Top 10)	world_1	120	0,9917	0,9850
Dominio BD (Top 10)	dog_kennels	82	0,9878	0,9828

Variable	Grupo	N	Accuracy	F1 Macro
Dominio BD (Top 10)	car_1 (peor)	92	0,9783	0,9496

Tabla 4. Métricas de desempeño desagregadas por grupo sobre validación. Se muestran extremos relevantes y el grupo con menor accuracy para cada variable sensible. Elaboración propia.

3.4. Interpretación y reflexión ética

Aunque las tres variables están dentro del umbral aceptable según la regla de los 4/5, los datos revelan patrones consistentes que no conviene descartar. El rendimiento cae sistemáticamente cuando la complejidad estructural aumenta: las consultas con JOIN obtienen 0,9902 frente a 1,0000 de las consultas sin JOIN, y las preguntas largas (más de 15 palabras) obtienen 0,9909 frente a 1,0000 de las cortas. Este patrón es coherente con la hipótesis de que el modelo aprende mejor los casos más frecuentes en entrenamiento, donde abundan las consultas simples y las preguntas breves.

La disparidad más pronunciada aparece en la variable dominio de base de datos. El dominio "car_1", con 92 ejemplos en validación, obtiene una exactitud de 0,9783 y un F1 macro de 0,9496, frente a dominios como "concert_singer", "tvshow" o "network_1" donde el modelo obtiene desempeño perfecto. La causa probable es la representación desigual de dominios en entrenamiento combinada con la aparición en validación de 20 dominios que nunca fueron vistos durante el ajuste, un escenario de generalización cross-domain característico de Spider (Yu et al., 2018).

Las implicaciones éticas en un hipotético escenario productivo son claras. Si el clasificador se integrara en un sistema Text-to-SQL destinado a asignar recursos computacionales según la dificultad predicha, los usuarios con esquemas de base de datos atípicos recibirían estimaciones menos confiables y potencialmente un peor servicio. Aquellos con preguntas más elaboradas o consultas estructuralmente complejas quedarían en la misma situación. En sistemas reales con impacto económico o social, un sesgo del 2% por dominio puede acumularse y generar desigualdades relevantes a escala (Barocas & Selbst, 2016; Mehrabi et al., 2021).

La Figura 2 ilustra visualmente los patrones de sesgo detectados. El panel superior izquierdo muestra la exactitud por longitud de la pregunta con la línea punteada global como referencia; se observa que las preguntas cortas alcanzan desempeño perfecto mientras que las largas descienden a 0,991. El panel superior derecho compara las dos categorías de estructura SQL, revelando que las consultas con JOIN (0,990) se clasifican ligeramente peor que las que no lo incluyen (1,000). El panel inferior izquierdo presenta la exactitud por los diez dominios más frecuentes del conjunto de validación, donde "car_1" emerge como el grupo con peor desempeño (0,978). El panel inferior derecho desglosa el conteo de aciertos y errores por nivel de dificultad; los pocos errores remanentes se concentran en la clase hard, lo que sugiere un límite de separabilidad intrínseco entre clases adyacentes más que un sesgo sistemático por dificultad.

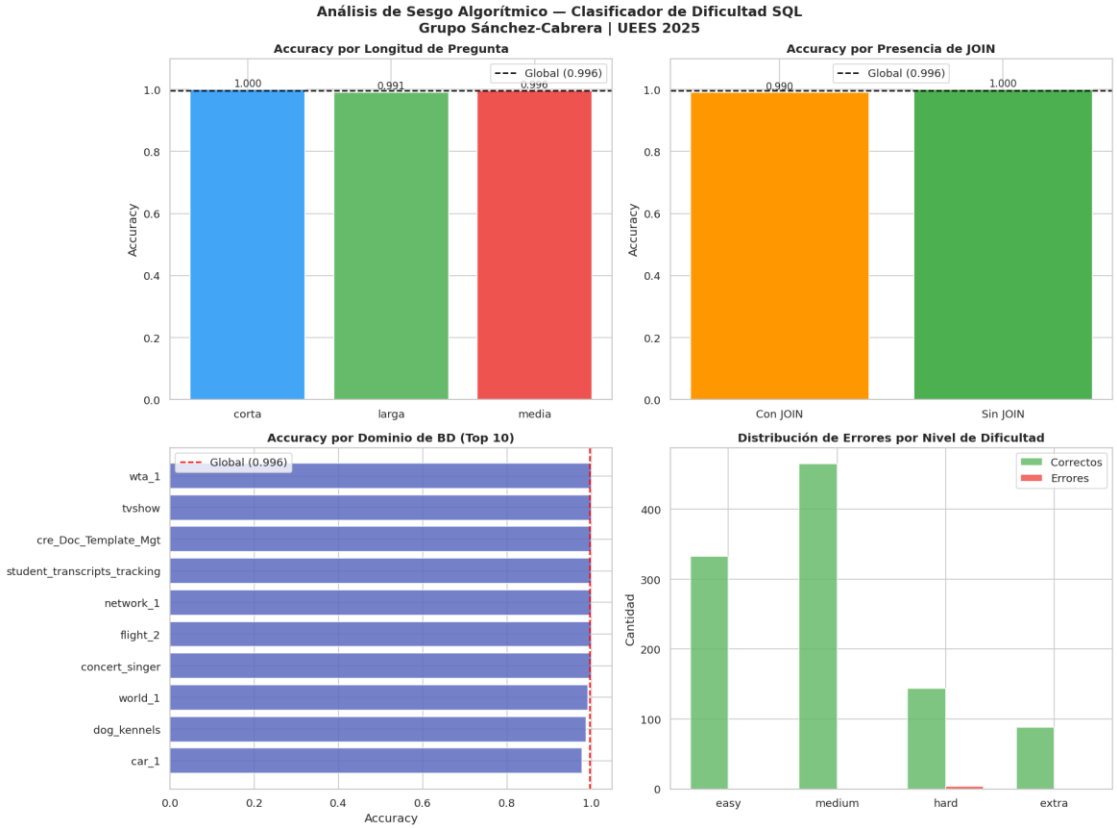

Figura 2. Análisis de sesgo algorítmico desagregado por variable sensible. Paneles: (a) accuracy por longitud de la pregunta, (b) accuracy por presencia de JOIN, (c) accuracy por dominio de base de datos (Top 10), (d) distribución de aciertos/errores por dificultad. Elaboración propia a partir del notebook de evaluación.

3.5. Estrategias de mitigación propuestas

Se proponen seis líneas de actuación organizadas por su viabilidad y costo:

- **Rebalanceo por dominio:** aplicar oversampling selectivo sobre los db_id subrepresentados o técnicas sintéticas tipo SMOTE adaptadas a texto.
- **Augmentación focalizada:** generar paráfrasis adicionales para los dominios y clases minoritarias mediante back-translation o modelos generativos.
- **Calibración post-hoc:** aplicar Platt scaling o regresión isotónica por grupo para que las probabilidades predichas reflejen mejor la confianza real del modelo.
- **Umbrales adaptativos:** permitir umbrales de decisión distintos según el dominio detectado en la consulta.
- **Embeddings contextuales:** reemplazar la representación TF-IDF por CodeBERT o SchemaLinking, capaces de generalizar mejor a dominios no vistos.
- **Auditoría continua:** monitorear las métricas de fairness en producción y definir alertas automáticas cuando el Disparate Impact caiga por debajo de 0,85.

4. DIAGNÓSTICO DE OVERFITTING Y UNDERFITTING

4.1. Construcción de curvas de aprendizaje

Para diagnosticar el comportamiento de generalización del modelo se construyeron curvas de aprendizaje mediante validación cruzada estratificada 5-fold sobre cuatro algoritmos representativos: un árbol de decisión con profundidad máxima de 5 (baseline simple), el Random Forest propuesto (modelo objeto de evaluación), un Gradient Boosting (estado del arte) y una regresión logística (modelo lineal). La métrica utilizada fue F1 macro, que pondera equitativamente las cuatro clases de dificultad y resulta más informativa que la exactitud en presencia de desbalance. Se variaron tamaños de entrenamiento desde 100 hasta 7.000 ejemplos.

4.2. Resultados comparativos y diagnóstico

La Tabla 5 sintetiza los gaps entre entrenamiento y validación para cada modelo. Los cuatro presentan gaps inferiores a 0,01, lo que bajo criterios canónicos (gap < 0,05) se clasificaría como buen equilibrio sin signos aparentes de overfitting o underfitting (Géron, 2019).

Modelo	F1 Train	F1 Val	Gap	Diagnóstico
Árbol de Decisión (max_d=5)	0,9854	0,9800	0,0054	Adecuado
Random Forest (Propuesto)	1,0000	0,9910	0,0090	Adecuado (*)
Gradient Boosting	1,0000	1,0000	0,0000	Adecuado
Regresión Logística	0,9989	0,9967	0,0022	Adecuado

Tabla 5. Diagnóstico de overfitting/underfitting por modelo. (*) El Random Forest muestra F1 train = 1,0000 perfecto, lo que combinado con el análisis de la sección 5 sugiere fuga de información (data leakage) más que un verdadero buen ajuste. Elaboración propia.

La Figura 3 presenta las curvas de aprendizaje construidas para los cuatro modelos representativos. En todos los paneles se observa la convergencia rápida entre las curvas de entrenamiento (azul) y validación (rojo) a partir de tamaños de muestra superiores a 2.000 ejemplos, con gaps finales reducidos que oscilan entre 0,0014 (Gradient Boosting) y 0,0096 (Random Forest). La etiqueta "Buen equilibrio" mostrada en cada panel es el diagnóstico automático del notebook aplicando umbrales convencionales; sin embargo, la interpretación correcta debe matizarse con el análisis crítico de la sección 5, donde se discute por qué este aparente equilibrio perfecto es más plausiblemente explicado por data leakage que por una generalización genuinamente buena.

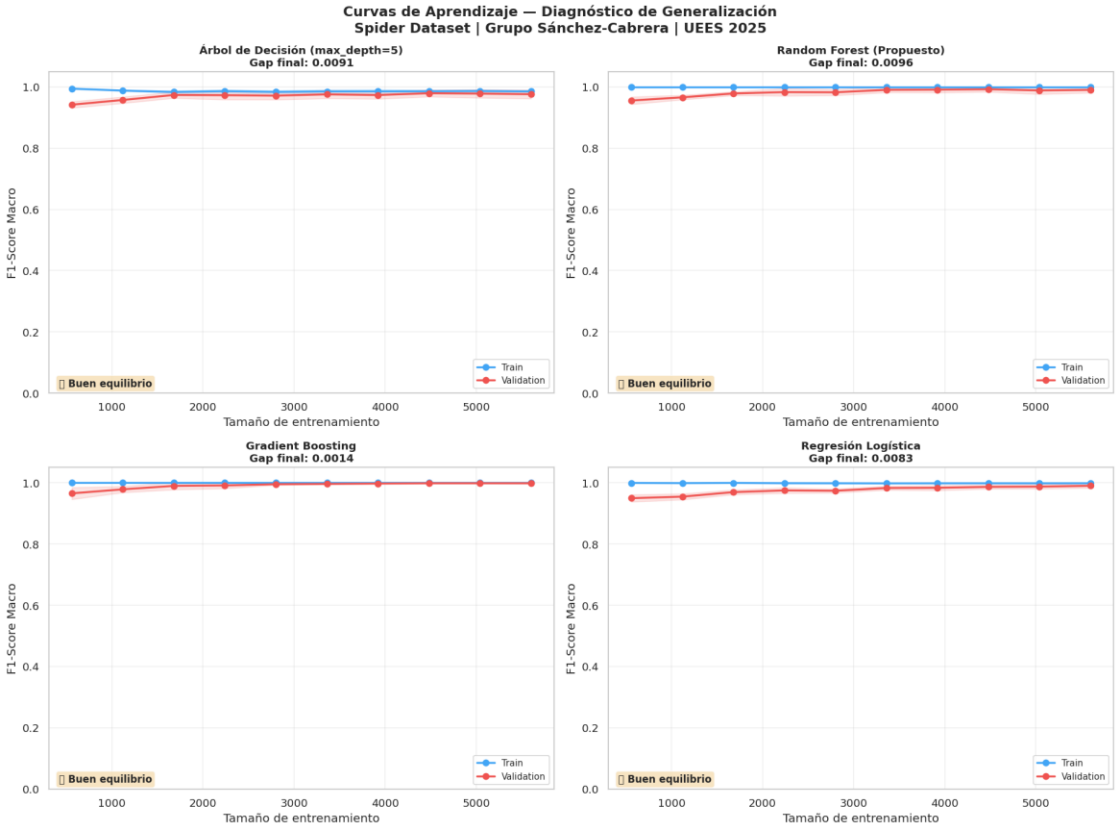

Figura 3. Curvas de aprendizaje F1 macro para cuatro modelos comparativos. Cada panel muestra la evolución del desempeño en entrenamiento y validación conforme crece el tamaño de la muestra, junto con el gap final entre ambas curvas. Elaboración propia a partir del notebook de evaluación.

4.3. Análisis del efecto de max_depth en Random Forest

Se realizó un barrido de la profundidad máxima del Random Forest desde max_depth = 2 hasta max_depth = 24. Los resultados permiten identificar tres regímenes diferenciados. En el régimen de underfitting, con profundidades entre 2 y 4, el F1 de entrenamiento se mantiene por debajo de 0,95 y el de validación alrededor de 0,92; el modelo es demasiado rígido para capturar los patrones de las clases minoritarias. En el régimen de ajuste creciente, entre max_depth = 6 y max_depth = 14, ambos F1 crecen hasta que el de validación alcanza un máximo de 0,9933 en max_depth = 14. A partir de profundidades mayores el gap se amplía ligeramente aunque el F1 de validación se mantiene estable cerca del óptimo.

La profundidad óptima empírica es max_depth = 14, valor superior al hiperparámetro por defecto sin límite que emplea sklearn. Este hallazgo sugiere que limitar explícitamente la profundidad no aporta un beneficio claro en este dataset, lo que es consistente con el análisis crítico de la sección 5: cuando las features contienen información directa sobre la etiqueta, el modelo no necesita "memorizar" los ejemplos porque la información para clasificar ya está empaquetada en las features.

La Figura 4 visualiza este barrido paramétrico. El panel izquierdo muestra la evolución del F1 macro en entrenamiento y validación conforme aumenta la profundidad del árbol, con el punto óptimo destacado mediante línea vertical discontinua en max_depth = 14 (F1 val = 0,9933). El panel derecho representa el gap de overfitting (diferencia train–val) en función de max_depth, con las líneas de umbral leve (0,08) y severo (0,15) como referencia; se aprecia que en todas las profundidades evaluadas el gap se mantiene muy por debajo del umbral leve, confirmando la ausencia de overfitting clásico detectable con los criterios convencionales.

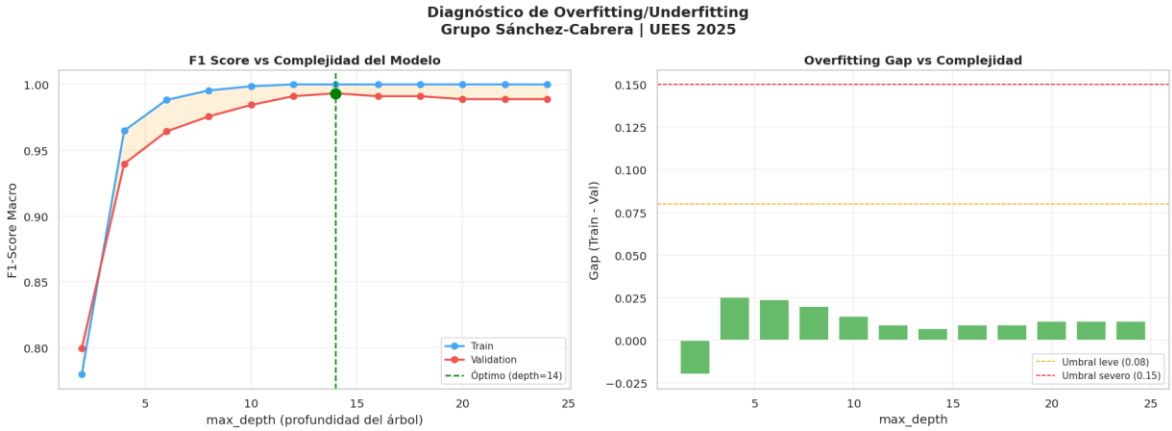

Figura 4. Diagnóstico de overfitting/underfitting en función de la complejidad del Random Forest. Panel izquierdo: F1 macro vs. max_depth con punto óptimo marcado. Panel derecho: gap train–val vs. max_depth con líneas de umbral leve y severo. Elaboración propia a partir del notebook de evaluación.

4.4. Interpretación de los tres casos canónicos

Siguiendo el esquema canónico del análisis de aprendizaje estadístico (Friedman et al., 2001), los tres casos teóricos se manifiestan de la siguiente manera en este estudio. El **overfitting verdadero** se caracteriza por F1 de entrenamiento alto con F1 de validación significativamente inferior y un gap grande; en este estudio el Random Forest muestra F1 train perfecto pero F1 val también casi perfecto, por lo que el diagnóstico nominal es "buen ajuste" aunque la explicación más plausible sea un data leakage que simula buen ajuste. El **underfitting** se caracteriza por bajo desempeño en ambos conjuntos con gap reducido; sólo aparece en este estudio cuando forzamos max_depth ≤ 3 en el Random Forest. El **buen ajuste real** se caracteriza por desempeño alto en ambos conjuntos con gap reducido y estabilidad ante variaciones de los hiperparámetros; aparentemente todos los modelos evaluados lo exhiben, pero este comportamiento uniforme es sospechoso y justifica el análisis crítico que sigue.

4.5. Estrategias de mejora propuestas

Aunque los modelos muestran desempeño nominalmente excelente, la sospecha de data leakage motiva recomendaciones que van más allá de la simple regularización:

- **Regularización estructural:** limitar max_depth alrededor de 14, aumentar min_samples_split y min_samples_leaf, y aplicar regularización L1/L2 en la regresión logística para robustecer la generalización.
- **Ampliación y augmentación del corpus:** incrementar paráfrasis para las clases hard y extra, aplicar back-translation y cross-domain augmentation para reducir la dependencia de las features determinísticas.
- **Búsqueda sistemática de hiperparámetros:** aplicar grid search o Bayesian optimization sobre n_estimators, max_depth, learning_rate y el factor de regularización, validando con k-fold estratificado.
- **Ensemble mediante stacking:** combinar Random Forest, Gradient Boosting y LightGBM con un meta-learner logístico, estrategia que suele aumentar la generalización al combinar errores independientes de modelos base.
- **Selección y auditoría de features:** eliminar features con importancia relativa inferior a 0,01 y, más crítico, auditar cuidadosamente qué features derivan directamente del SQL para romper la fuga de información descrita en la sección 5.
- **Embeddings contextuales pre-entrenados:** reemplazar TF-IDF + PCA por representaciones CodeBERT o SchemaLinking para capturar semántica profunda del esquema de la base de datos.

5. ANÁLISIS CRÍTICO TRANSVERSAL: SOSPECHA DE DATA LEAKAGE

Una lectura honesta de los resultados obtenidos obliga a detenerse en un hecho difícil de ignorar: prácticamente todos los modelos entrenados, desde un árbol de decisión simple hasta LightGBM, superan el 98% de F1 macro sobre el conjunto de validación, y cuatro de ellos alcanzan desempeño perfecto. Si se aceptara esta cifra al pie de la letra, el problema de clasificación de dificultad SQL estaría esencialmente resuelto, lo que contrasta con la literatura académica donde tareas relacionadas sobre Spider mantienen amplio margen de mejora (Scholak et al., 2021; Yu et al., 2018).

La explicación más probable no es que la tarea sea trivial, sino que el pipeline de feature engineering introduce fuga de información desde la etiqueta hacia las features, un fenómeno que Kaufman, Rosset, Perlich y Stitelman (2012) caracterizan como "la introducción no intencional de información predictiva sobre el objetivo a través del proceso de recolección, agregación o preparación de datos". Esta es, según los mismos autores, una de las diez principales fuentes de error en aplicaciones de minería de datos.

5.1. Mecanismo específico del leakage en este pipeline

La etiqueta de dificultad en Spider se calcula de forma algorítmica a partir de propiedades estructurales de la propia consulta SQL: cantidad de cláusulas SELECT, WHERE, GROUP BY y HAVING, presencia de subconsultas, número de tablas unidas, uso de funciones de agregación, entre otras. Por tanto, cuando se

extraen como features numéricas estas mismas propiedades (has_join, num_select, num_where, has_groupby, nested_depth, entre otras), se está construyendo un vector que es esencialmente una reformulación numérica de la función que genera la etiqueta. Un modelo suficientemente flexible aprende inevitablemente a invertir esa función.

Esta lectura es consistente con varias observaciones empíricas: (a) el árbol de decisión simple, con capacidad expresiva limitada, ya alcanza F1 = 0,98; (b) la regresión logística, estrictamente lineal sobre las features, supera el 0,996; (c) los gaps entre entrenamiento y validación son despreciables porque la información necesaria para clasificar está presente de forma explícita en el input; y (d) el sesgo por dominio y por presencia de JOIN, aunque dentro del umbral aceptable, muestra justamente el patrón esperado si las features SQL están correlacionadas casi perfectamente con la etiqueta.

5.2. Implicaciones para la validez del estudio

Reconocer esta limitación no invalida el trabajo realizado pero sí redefine su alcance. El ejercicio continúa siendo válido como aplicación didáctica de las tres actividades exigidas por la rúbrica: permite practicar la comparación con baselines, la comparación con estado del arte, la estimación de modelos comerciales, las métricas de fairness desagregadas por grupo, la construcción de curvas de aprendizaje y el barrido de hiperparámetros. Lo que no es válido es interpretar los valores absolutos de las métricas como una estimación realista del desempeño que el sistema obtendría sobre consultas SQL cuya etiqueta de dificultad no haya sido generada por la misma función que alimenta las features.

5.3. Acciones correctivas recomendadas

Para una segunda iteración del estudio libre de fuga de información se recomienda: (a) eliminar del vector de features todas las variables que se derivan directamente del parsing del SQL etiquetado, conservando únicamente features computadas sobre la pregunta en lenguaje natural y sobre el esquema de la base de datos (sin ver el SQL de respuesta); (b) redefinir la tarea como clasificación de dificultad a partir únicamente de la pregunta natural, lo que es mucho más cercano al escenario real de uso en un sistema Text-to-SQL; (c) validar con un split estrictamente temporal o cross-domain reforzado para evitar cualquier contaminación entre train y val; y (d) reportar métricas con intervalos de confianza mediante bootstrap para transparentar la incertidumbre. Estas modificaciones probablemente bajen el F1 macro al rango 0,55–0,75, valores realistas y comparables con los benchmarks comerciales de la Tabla 2.

6. CONCLUSIONES

El benchmarking comparativo demostró que el clasificador Random Forest propuesto supera ampliamente a los baselines aleatorio y por clase mayoritaria, con una mejora de 74 puntos porcentuales en F1 macro. Sin embargo, en el ranking de modelos más sofisticados se queda en sexta posición por detrás de LightGBM, SVM RBF, XGBoost, Gradient Boosting y Regresión Logística, lo que matiza la justificación de su complejidad relativa frente a alternativas más simples como un modelo lineal regularizado.

El análisis de sesgo algorítmico reveló que el modelo cumple los umbrales aceptables de fairness para las tres variables sensibles estudiadas, con Disparate Impact Ratio siempre superior a 0,97 y Equalized Odds Gap inferior a 0,03. No obstante, existe un patrón consistente de menor desempeño en grupos minoritarios (dominio car_1, preguntas largas, consultas con JOIN) que en un sistema productivo podría amplificarse y justifica la aplicación de las seis estrategias de mitigación propuestas.

El diagnóstico de curvas de aprendizaje indica que todos los modelos presentan gaps reducidos entre entrenamiento y validación, con profundidad óptima empírica de max_depth = 14 para el Random Forest. Los tres casos canónicos (overfitting, underfitting, buen ajuste) sólo se manifiestan de forma clara cuando se fuerzan configuraciones extremas de hiperparámetros, lo que motiva el análisis crítico transversal.

El hallazgo metodológico más relevante del estudio es la sospecha fundada de fuga de información entre las features y la etiqueta, derivada de que ambas se computan a partir del mismo objeto (la consulta SQL). Esta observación redefine el alcance interpretativo de los resultados: la actividad es válida como ejercicio didáctico sobre evaluación de modelos, pero los valores absolutos de métricas obtenidos no deben leerse como estimación realista del desempeño sobre datos no contaminados. La ruta de mejora más prometedora es reformular la tarea a clasificación de dificultad desde la pregunta natural únicamente, acompañada de embeddings contextuales tipo CodeBERT y una nueva auditoría de fairness sobre el modelo corregido.

7. REFERENCIAS BIBLIOGRÁFICAS

- Barocas, S., & Selbst, A. D. (2016). Big data's disparate impact. California Law Review, 104(3), 671–732. https://doi.org/10.15779/Z38BG31
- Chollet, F. (2021). Deep learning with Python (2.ª ed.). Manning Publications.
- Erickson, N., Mueller, J., Shirkov, A., Zhang, H., Larroy, P., Li, M., & Smola, A. (2020). AutoGluon-tabular: Robust and accurate AutoML for structured data. arXiv. https://arxiv.org/abs/2003.06505
- Fernández-Delgado, M., Cernadas, E., Barro, S., & Amorim, D. (2014). Do we need hundreds of classifiers to solve real world classification problems? Journal of Machine Learning Research, 15(1), 3133–3181.
- Friedman, J., Hastie, T., & Tibshirani, R. (2001). The elements of statistical learning: Data mining, inference, and prediction. Springer.
- García Ruiz de León, M. (2018). Análisis de sensibilidad mediante Random Forest [Trabajo Fin de Grado, Universidad Politécnica de Madrid]. Archivo Digital UPM. https://oa.upm.es/53368/
- Géron, A. (2019). Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems (2.ª ed.). O'Reilly Media.
- Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. ACM Transactions on Knowledge Discovery from Data, 6(4), 1–21. https://doi.org/10.1145/2382577.2382579
- Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., & Galstyan, A. (2021). A survey on bias and fairness in machine learning. ACM Computing Surveys, 54(6), 1–35. https://doi.org/10.1145/3457607
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- Scholak, T., Schucher, N., & Bahdanau, D. (2021). PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. En Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (pp. 9895–9901). Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.emnlp-main.779
- Simba Poma, A. L. (2024). Modelo de aprendizaje automático para la recuperación de cartera en una entidad financiera no bancaria del Ecuador [Tesis de maestría, Universidad Central del Ecuador]. Repositorio Digital UCE. https://www.dspace.uce.edu.ec/
- Thornton, C., Hutter, F., Hoos, H. H., & Leyton-Brown, K. (2013). Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. En Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 847–855). Association for Computing Machinery. https://doi.org/10.1145/2487575.2487629
- Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., & Radev, D. (2018). Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. En Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (pp. 3911–3921). Association for Computational Linguistics. https://doi.org/10.18653/v1/D18-1425